

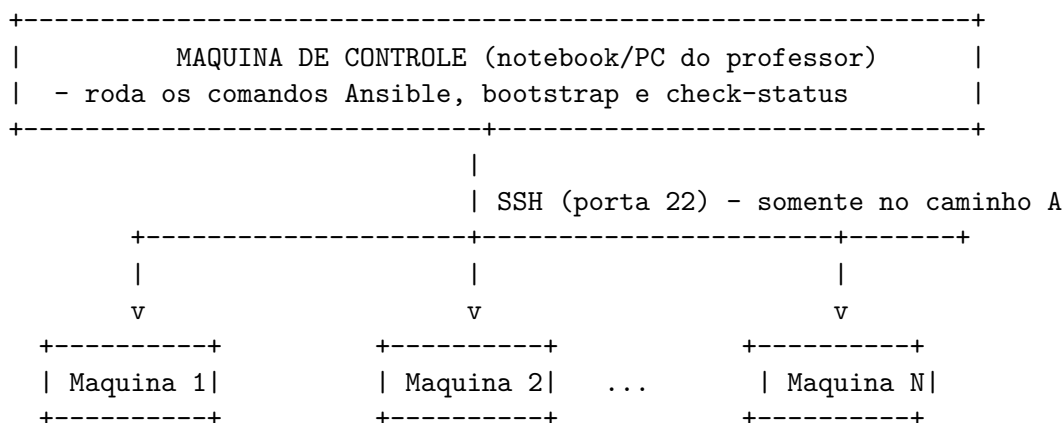
Sumário

PenguinLab — Guia Prático de Implantação	2
1. Visão geral do fluxo	2
2. Antes de começar: decidir o caminho	2
3. Preparação das máquinas (igual nos dois caminhos)	3
4. A rede do laboratório: como saber se o Caminho A vai funcionar	3
5. Caminho A — provisionamento com rede (Ansible)	4
6. Caminho B — provisionamento sem rede (pendrive)	6
7. Checklist completo de verificação pós-deploy	7
8. Manutenção	8
9. Rollback — desfazer o provisionamento	8
10. Referência rápida de comandos	9

PenguinLab — Guia Prático de Implantação

Passo a passo completo para colocar o PenguinLab em funcionamento em um laboratório de informática escolar, partindo de máquinas com instalação padrão do Linux Mint (ou Ubuntu). O guia cobre os dois cenários: **com rede** (provisionamento em lote via Ansible) e **sem rede** (provisionamento manual via pendrive).

1. Visão geral do fluxo



Há dois papéis:

- **Máquina de controle (notebook do professor):** onde o Ansible é instalado e de onde partem os comandos de provisionamento e verificação.
- **Máquinas do laboratório (hosts):** as máquinas dos alunos. Precisam apenas do Linux Mint instalado, da conta **professor** e, no caminho com rede, do OpenSSH server ativo. **Nenhuma máquina do laboratório precisa de Ansible** — elas só precisam de `python3`, que o Mint já inclui por padrão.

2. Antes de começar: decidir o caminho

	Caminho A — com rede	Caminho B — sem rede (pendrive)
Precisa de rede entre o notebook e as máquinas?	Sim	Não
Tempo para 14 máquinas	Minutos (em paralelo)	Cerca de 1 h (3–5 min por máquina)
Verificação em lote (<code>check-status.sh</code>)	Sim	Manual
Reprovisionar depois	Rode o playbook de novo	Volte com o pendrive

Regra prática: se o notebook consegue “ver” as máquinas na rede (mesma faixa de IP), use o Caminho A. Se não, use o Caminho B. A seção 4 ensina como descobrir isso em um minuto.

Os dois caminhos aplicam exatamente a mesma configuração — a única diferença é o transporte do script (SSH em lote ou pendrive manual).

3. Preparação das máquinas (igual nos dois caminhos)

3.1 Instalar o sistema

1. Instale o **Linux Mint** (recomendado) ou **Ubuntu**, com instalação padrão e sem customização.
 - Máquinas com pouca memória (até 4 GB de RAM): use o **Mint XFCE**.
 - Máquinas mais parrudas: use o **Mint Cinnamon**.
2. Durante a instalação, crie a conta administrativa **professor** (pode ser outro nome, desde que o mesmo em todas as máquinas).
 - **NUNCA** chame essa conta de **aluno** — o provisionamento bloqueia o sudo dela e você se trancaria fora.

3.2 Ativar o OpenSSH server (obrigatório para o Caminho A)

No primeiro boot de cada máquina, logado como **professor**, execute uma única vez:

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

O Mint/Ubuntu desktop **não ativa** o SSH por padrão. Sem esse passo, o notebook não consegue se conectar às máquinas — e o `bootstrap-ssh.sh` vai avisar exatamente isso (ele detecta a porta 22 fechada, mostra a orientação acima e pula a máquina).

3.3 Catalogar as máquinas (recomendado em qualquer cenário)

Monte uma tabela simples para organizar o laboratório:

#	Hostname	IP	RAM	Disco	Observação
1	penguinlab-01	192.168.0.21	8 GB	256 GB	—
...	...	...	...	...	...

Os IPs podem ser obtidos pelo roteador (tabela de DHCP) ou pelo `--descobrir` do bootstrap (Caminho A).

4. A rede do laboratório: como saber se o Caminho A vai funcionar

A pergunta-chave **não** é “cabo ou wifi” — é **se o notebook e as máquinas estão na mesma rede IP** (mesma faixa de endereço, com rota entre eles).

Cenário A.1 — notebook e máquinas no mesmo roteador (funciona)

```
[roteador da escola]
wifi --> notebook (192.168.0.15)
cabo --> máquinas (192.168.0.21-.34)    <- mesma faixa
```

Se o wifi e o cabo saem do mesmo roteador, tudo está na mesma faixa (ex.: 192.168.0.x). O notebook no wifi **encontra normalmente** as máquinas cabeadas — a varredura do bootstrap funciona por TCP na porta 22 e não se importa se a origem é wifi ou cabo. **Este é o cenário mais comum em escolas pequenas.**

Cenário A.2 — redes separadas (não funciona sem ajuste)

```
wifi (10.10.x.x) --> notebook          <- sub-redes diferentes
cabo (192.168.0.x) --> máquinas
```

Quando o wifi e o cabo estão em faixas diferentes (ou em VLANs separadas com firewall), o notebook **não enxerga** as máquinas: o `--descobrir` varre uma única faixa e não há rota entre as sub-redes. Nesse caso, as opções são:

Opção	Como fazer
Plugar o notebook no cabo	Use um adaptador USB-C -> RJ45 (ou a porta RJ45 do notebook) em uma tomada do switch do laboratório. O notebook entra na mesma rede das máquinas
Travel router em modo bridge	Um roteador de viagem ligado por cabo ao switch do laboratório, com o notebook conectado no wifi dele. O notebook entra na rede do laboratório como se estivesse no cabo
Pendrive (Caminho B)	Sem rede, use o provisionamento manual — funciona sempre

Se a escola usa **VLANs segmentadas** (rede de alunos separada da rede de professores), mesmo plugging o notebook no cabo você pode ficar em uma rede que não conversa com a outra. Nesse caso, o travel router ou a liberação de rota pelo TI da escola resolve.

Verificação em 1 minuto

```
# 1. No notebook, veja sua faixa de rede
ip -4 addr show | grep inet
ip route | grep default          # ex.: 192.168.0.15/24 -> faixa 192.168.0

# 2. Em uma máquina do laboratório, veja o IP dela
hostname -I                     # ex.: 192.168.0.21

# 3. Compare os três primeiros octetos
#   Iguais (ex.: 192.168.0.x nos dois) -> mesmo cenário A.1, pode prosseguir
#   Diferentes                          -> cenário A.2, use uma das opções acima

# 4. Teste direto (troque pelo IP real)
ping 192.168.0.21              # respondeu? então está tudo certo
```

5. Caminho A — provisionamento com rede (Ansible)

5.1 Preparar a máquina de controle (notebook)

```
# Instalar o Ansible
sudo apt install ansible

# Garantir uma chave SSH (cria uma se não existir)
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
```

```
# Entrar na pasta do projeto
cd ~/projetos/PenguinLab/ansible
```

Todos os comandos deste caminho são executados a partir da pasta `ansible/`.

5.2 Catalogar e preparar o SSH (bootstrap)

```
./bootstrap-ssh.sh --descobrir 192.168.0 # troque pela faixa da sua rede
```

Ou, se você já sabe os IPs:

```
./bootstrap-ssh.sh 192.168.0.21 192.168.0.22 192.168.0.23
```

O que o bootstrap faz em cada máquina: 1. Verifica se a porta 22 está aberta — se estiver fechada, **avisa a orientação de ativar o SSH e pula a máquina** (o lote continua). 2. Garante `openssh-server` e `python3` (no Mint já existem). 3. Copia a **chave pública** do notebook para a conta `professor` (pede a senha do professor uma vez por máquina). 4. Valida o login sem senha.

Ao final, o notebook entra em todas as máquinas **sem pedir senha**. O progresso fica registrado em `bootstrap-ssh.log` (na pasta `ansible/`).

5.3 Preencher o inventário

Edite `ansible/inventory.ini` com os IPs reais encontrados:

```
[penguinlab]
penguinlab-01 ansible_host=192.168.0.21
penguinlab-02 ansible_host=192.168.0.22
# ... demais máquinas
```

```
[all:vars]
ansible_user=professor
```

- Para mais ou menos de 14 máquinas, é só adicionar ou remover linhas — não há limite fixo.
- `ansible_user` deve ser sempre a conta administrativa (`professor`), **nunca** aluno.

5.4 Definir a senha do usuário aluno

```
export PENGUINLAB_PASSWORD='aluno'
```

- Este projeto usa senha uniforme `aluno` para facilitar o uso pelas crianças (ver seção 3 do README).
- Se preferir outra senha, basta trocar o valor — o playbook envia a variável para dentro do script remoto, e a senha é aplicada de fato na máquina (em toda execução).

5.5 Testar em uma única máquina

```
ansible-playbook -i inventory.ini playbook.yml --limit penguinlab-01 \
-e "penguinlab_password=$PENGUINLAB_PASSWORD"
```

O playbook copia o script para a máquina, executa como root (a saída colorida do script aparece no seu terminal) e remove o arquivo temporário. **Verifique a máquina de teste manualmente** (seção 7) antes de liberar o laboratório inteiro.

5.6 Aplicar em todas as máquinas

```
ansible-playbook -i inventory.ini playbook.yml \
-e "penguinlab_password=$PENGUINLAB_PASSWORD"
```

Com `forks = 14` no `ansible.cfg`, todas as máquinas são provisionadas em paralelo. Se uma falhar, ela aparece como `FAILED` e as demais continuam — o resumo final mostra o status por máquina.

5.7 Reiniciar as máquinas

O playbook **não reinicia** por padrão (recomendado, para você conferir a máquina de teste antes). Para reiniciar todas de uma vez, do notebook:

```
for ip in $(grep -oE 'ansible_host=[0-9.]+' inventory.ini | cut -d= -f2); do
ssh professor@$ip "sudo reboot" &
done; wait
```

Ou descomente a tarefa de `reboot` no fim do `playbook.yml` (cuidado: reinicia todas simultaneamente).

5.8 Verificar (check-status)

```
./check-status.sh
```

Saída esperada: linhas `[OK]` verdes por checagem em cada máquina e, no final, **Resumo: 14/14** maquinas totalmente `OK`. (ou use `./check-status.sh <ip>` para uma máquina isolada).

6. Caminho B — provisionamento sem rede (pendrive)

Funciona em qualquer situação — não depende de rede nenhuma. Você só perde o lote automático: vai máquina por máquina.

6.1 Preparar o pendrive (uma vez)

1. Copie o arquivo `provision-penguinlab.sh` para a raiz de um pendrive.
2. (Opcional) Copie também o `README.md`, para consultar o checklist durante a operação.

6.2 Em cada máquina, passo a passo

a) Ligue a máquina e logue como **professor**. Conecte o pendrive — o Mint monta automaticamente em `/media/professor/<NOME_DO_PENDRIVE>` (confira com `lsblk` ou `ls /media/professor/`).

b) Abra um terminal e execute:

```
cd /media/professor/<NOME_DO_PENDRIVE>/
sudo bash provision-penguinlab.sh
```

Detalhe importante sobre a senha do aluno: o padrão do script é a senha **aluno** (uniforme, fácil para as crianças) — portanto o comando acima já está correto, e o aviso amarelo de “senha padrão” é esperado.

Se algum dia você quiser usar outra senha, o valor precisa ser passado **junto do sudo**, porque o `sudo` apaga variáveis de ambiente por padrão:

```
sudo PENGUINLAB_PASSWORD='outra-senha' bash provision-penguinlab.sh
```

c) Acompanhe a execução: linhas verdes [+] para cada etapa (DNS -> Firefox -> usuário -> SSH -> auto-login), amarelas [!] para avisos e vermelhas [*] para erros. No final, aparece o resumo “Provisionamento Concluído”. O log completo fica em `/var/log/penguinlab-provision.log` na própria máquina.

d) Reinicie:

```
sudo reboot
```

e) Verificação rápida (a primeira máquina merece a checagem completa da seção 7):

Conferir	Como	Esperado
Auto-login	Ligar de novo	Entra direto no desktop do aluno
DNS filtrado	<code>resolvectl status</code>	1.1.1.3 / 1.0.0.3
Firefox	Abrir e ver <code>about:policies</code>	Restrições ativas
aluno sem sudo	<code>sudo -l -U aluno</code>	“not allowed”

f) Desligue o pendrive e siga para a próxima máquina. Como cada máquina é independente, um erro em uma delas é corrigido simplesmente rodando o script de novo (ele é idempotente).

Dica: se depois da implantação por pendrive você quiser voltar ao fluxo de rede (por exemplo, para rodar o `check-status.sh` em lote), basta ativar o SSH nas máquinas (`sudo apt install -y openssh-server && sudo systemctl enable --now ssh`), copiar a chave com `ssh-copy-id professor@<ip>` e usar o `check-status` — o provisionamento em si já terá sido feito.

7. Checklist completo de verificação pós-deploy

Vale aplicar em uma máquina de amostra (ou em todas, com o `check-status.sh` no Caminho A):

Verificação	Comando (na máquina)	Esperado
DNS ativo	<code>resolvectl status</code>	1.1.1.3 / 1.0.0.3 como DNS
DNS sobre TLS	<code>resolvectl status</code>	DNSOverTLS: opportunistic
NetworkManager respeita	<code>grep dns /etc/NetworkManager/conf.d/99-penguinlab-dns.conf</code>	<code>dns=systemd-resolved</code>
Firefox restrito	<code>about:policies</code> no Firefox	Restrições visíveis
Firefox sem DoH próprio	<code>about:policies -> DNSOverHTTPS</code>	Enabled: false, Locked: true
aluno sem sudo	<code>sudo -l -U aluno</code> (como professor)	“not allowed”
aluno não loga via SSH	<code>ssh aluno@<ip></code> (de outra máquina)	Conexão recusada
Ações administrativas bloqueadas	<code>pkexec nm-connection-editor</code> (como aluno)	Pede senha de admin ou falha
Instalação de pacotes bloqueada	<code>pkexec apt install hello</code> (como aluno)	Falha / pede senha
Shell do aluno livre	<code>su - aluno -> echo \$SHELL</code>	<code>/bin/bash</code>

Verificação	Comando (na máquina)	Esperado
Auto-login do aluno	<code>grep autologin-user /etc/lightdm/lightdm.conf</code>	<code>autologin-user=aluno</code>

8. Manutenção

8.1 Adicionar uma nova máquina

1. Instale o Mint, crie o `professor` e ative o `sshd`.
2. `./bootstrap-ssh.sh 192.168.0.NN`
3. Adicione a linha no `inventory.ini`.
4. `ansible-playbook -i inventory.ini playbook.yml --limit penguinlab-NN -e "penguinlab_password=$PENGUINLAB_PASSWORD"`
5. Reinicie a máquina.

8.2 Reprovisionar (atualizar configurações)

O script é idempotente: rode o playbook (ou o pendrive) de novo após alterar o repositório. Configurações existentes são atualizadas sem duplicação, e a **senha do aluno é reaplicada** a cada execução.

8.3 Trocar a senha do aluno

Basta reprovisionar com outro `PENGUINLAB_PASSWORD`:

```
export PENGUINLAB_PASSWORD='nova-senha'
ansible-playbook -i inventory.ini playbook.yml \
  -e "penguinlab_password=$PENGUINLAB_PASSWORD"
```

8.4 (Opcional) Desligar o bloqueio de tela

Como o `aluno` tem autologin, a senha dele só é usada quando a tela é bloqueada. Se quiser que as crianças **nunca** precisem digitar senha:

- Mint Cinnamon: Configurações do Sistema -> Proteção de tela -> desmarcar **Bloquear a tela quando suspender/proteger**.
- Ou force via política na máquina, se elas forem modificadas em todas.

8.5 (Opcional) Proibir a sessão de convidado

Se a tela de login (`greeter`) aparecer com a opção “Sessão de convidado”, remova-a para evitar sessões fora da conta `aluno`:

```
sudo sh -c 'echo "allow-guest=false" >> /etc/lightdm/lightdm.conf'
```

9. Rollback — desfazer o provisionamento

Para reverter uma máquina ao estado anterior:

```
# 1. Remover o bloqueio de sudo
rm -f /etc/sudoers.d/penguinlab-aluno
visudo -c # validar antes de continuar

# 2. Remover as restrições de polkit
rm -f /etc/polkit-1/rules.d/90-penguinlab-restrict.rules
systemctl restart polkit

# 3. Remover o bloqueio de SSH do aluno
rm -f /etc/ssh/sshd_config.d/90-penguinlab-no-ssh-aluno.conf
sshd -t && (systemctl reload sshd 2>/dev/null || systemctl reload ssh 2>/dev/null || true)

# 4. Restaurar o DNS padrão
rm -f /etc/NetworkManager/conf.d/99-penguinlab-dns.conf
if [ -f /etc/systemd/resolved.conf.penguinlab.bak ]; then
    mv /etc/systemd/resolved.conf.penguinlab.bak /etc/systemd/resolved.conf
else
    rm -f /etc/systemd/resolved.conf
fi
systemctl restart systemd-resolved NetworkManager

# 5. Remover as políticas do Firefox
rm -f /etc/firefox/policies/policies.json

# 6. (Opcional) Remover o usuário aluno - remove também a home
userdel -r aluno 2>/dev/null || true
```

10. Referência rápida de comandos

```
# Caminho A - com rede
sudo apt install ansible
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
export PENGUINLAB_PASSWORD='aluno'
cd ansible/
./bootstrap-ssh.sh --descobrir 192.168.0 # descobre + prepara SSH (precisa do sshd ativo)
# edite inventory.ini com os IPs reais
ansible-playbook -i inventory.ini playbook.yml --limit penguinlab-01 \
    -e "penguinlab_password=$PENGUINLAB_PASSWORD" # teste em 1 máquina
ansible-playbook -i inventory.ini playbook.yml \
    -e "penguinlab_password=$PENGUINLAB_PASSWORD" # aplicar em todas
./check-status.sh # verificar

# Caminho B - sem rede (pendrive)
# copie provision-penguinlab.sh para o pendrive e, em cada máquina:
cd /media/professor/<NOME_DO_PENDRIVE>/
sudo bash provision-penguinlab.sh
sudo reboot
```